Finding the differences between two pictures

Reproducing the TinyCD model and checking its published score

The short version. We took a very small change-detection model called **TinyCD** (about 285,000 settings, a 1.3 MB file), ran the authors' own test program on their own test pictures, and got **F1 = 91.07**. The paper reports **91.05**. **So the published result was reproduced successfully.**

One correction to a common reading of these numbers. 91.07 is *not* "way better" than 91.05. The gap is 0.02 points. Repeating the test on a slightly different random draw of the same pictures moves the score by about ±0.4 points, so the two numbers are, for every practical purpose, **the same**. The honest claim is: **"we matched the paper"**, not "we beat it". Matching is exactly what a reproduction is supposed to show.

At a glance

Question	Answer
What does the model do?	Takes two photos of the same place (before / after) and paints the parts that changed.
What was tested?	2,048 before/after aerial-photo pairs (the official LEVIR-CD test set), each 256 × 256 pixels.
Paper's score (F1)	91.05
Score we measured (F1)	**91.07** (95% range 90.63 - 91.47)
Reproduced?	**Yes.** All four headline scores are within 0.07 points of the paper (page 4).
Overall pixel accuracy	99.11% (but this number is flattering - see page 3)
Speed / size	about 18-21 ms per pair on an old GTX 1080 Ti graphics card, about 55 MB of GPU memory, 1.28 MB model file
Where it works	Buildings that appeared or disappeared in aerial photos that line up with each other.
Where it does not	Very tiny changes; a different dataset (SYSU-CD) where it found almost nothing; anything outside aerial buildings was not tested.
Can a client use it commercially?	**Not without written permission** from the authors (code is research / non-commercial only).

What "reproduced" means here. We re-ran the *released, already-trained* model on the official test pictures using the authors' own, unmodified test script. We did **not** train the model again from scratch, and we did **not** reproduce the paper's second dataset (WHU-CD) - the reasons are on page 9.

1. What is TinyCD, in plain words?

Imagine two satellite photos of the same neighbourhood taken a few years apart. A person can spot the new houses by flipping between the photos. **Change detection** asks a computer to do the same: given photo **A** (before) and photo **B** (after), produce a black-and-white picture where **white means "this spot changed"**.

A naive approach is to subtract one photo from the other, pixel by pixel. That fails in practice, because shadows, season, sun angle and image quality change *everything*, not just the buildings. A trained model learns what a "real" change (a new building) looks like and ignores the rest.

TinyCD is a trained model built to be **small**. The table shows how small it is.

Property	TinyCD
Number of learned settings (parameters)	285,128
File size of the trained model	1.28 MB
Input	Two colour pictures of the same size (it was trained on 256 × 256 tiles)
Output	A 'change probability' for every pixel, from 0 (no change) to 1 (change). Anything above 0.5 is painted white.
Trained on	LEVIR-CD: building changes seen from the air
Source	Codegoni, Lombardi & Ferrari (2022), published in Neural Computing and Applications; code at github.com/AndreaCodegoni/Tiny_model_4_CD

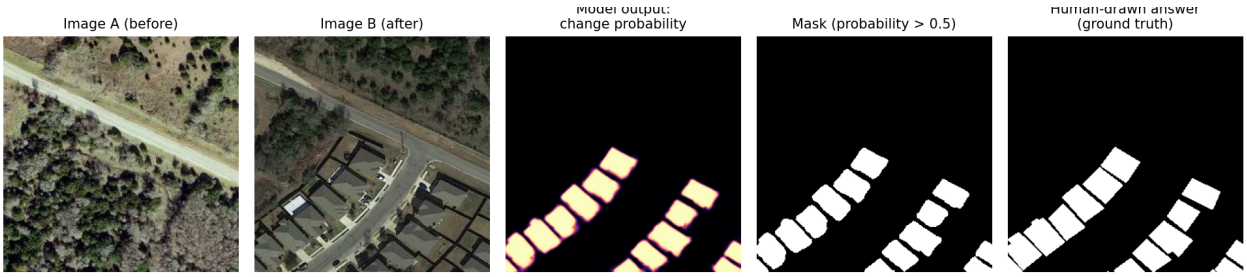


Figure 1 - A real test pair (LEVIR-CD, file test_01777). The model sees only the first two pictures. The last picture is the human-drawn answer, which is used only to grade the model afterwards.

Notice how closely the white shapes in the model's mask follow the human-drawn answer. That is what a good score looks like.

Why we picked it

The task was to find **one lightweight model** that takes two images and outputs a pixel-level change mask, with public code and a downloadable trained file, that can run on a modest graphics card. TinyCD fits all of those requirements. We searched for a better lightweight alternative and found none with public, ready-to-use weights that was clearly stronger, so we kept TinyCD. (We did not run a head-to-head test of other models.)

2. How we measure the score (the 'evaluation matrix')

To grade the model we compare its white-and-black mask against the human-drawn answer, **pixel by pixel**. Each pixel falls into one of four groups:

	Model says changed	**Model says not changed**
Human says **changed**	**True positive** - correct catch	**False negative** - a miss
Human says **not changed**	**False positive** - a false alarm	**True negative** - correctly left alone

From these four counts we get the scores used throughout the report:

Score	**Plain-English meaning**	**Our value**
Precision	Of everything the model painted white, how much was really a change? (low = many false alarms)	92.65%
Recall	Of all the real changes, how much did the model find? (low = many misses)	89.54%
F1	One number balancing precision and recall. **This is the headline score.**	**91.07%**
IoU	How much the model's white area overlaps the human white area, divided by their combined area. A stricter score than F1.	83.60%
Accuracy	Share of *all* pixels that are right. Misleading here (see below).	99.11%

Why accuracy alone is a trap. In these photos only about 5% of the pixels are changes (6.8 million of 134 million). A lazy model that painted *nothing* white would still be about 95% 'accurate'. That is why change detection uses **F1** and **IoU**, which only reward the model for actually finding the changes.

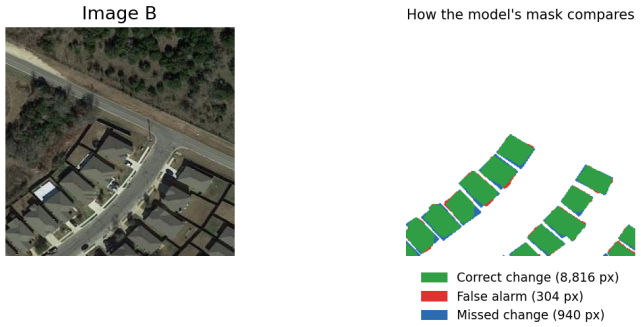


Figure 2 - Grading one pair. Green = correct, red = false alarm, blue = missed. Nearly everything is green; the errors are thin edges around the buildings (8,816 correct pixels vs 304 false alarm and 940 missed).

The settings we used

- **Cut-off:** a pixel counts as 'changed' when the model's probability is above 0.5. This is the authors' own setting. We did **not** adjust it using the test pictures.
- **Pooling:** the four counts are added up over all 2,048 test pairs before computing the scores, exactly as in the authors' test program.
- **Error bars:** we reshuffled the test pairs 1,000 times (resampling with replacement) to see how much the score would wobble. The F1 score stays between 90.63 and 91.47 in 95% of reshuffles. This is the 'give or take' to keep in mind when comparing 91.07 with 91.05.

3. The score we reproduced

We ran the authors' unmodified **test_ondata.py** with their released file **levir_best.pth** on the official LEVIR-CD test pictures. Here is the comparison:

Score	Paper	Our run	Difference	Our 95% range
Precision	92.68	92.65	-0.03	92.31 - 93.02
Recall	89.47	89.54	+0.07	88.76 - 90.27
F1 (headline)	**91.05**	**91.07**	**+0.02**	90.63 - 91.47
IoU	83.57	83.60	+0.03	82.87 - 84.28
Overall accuracy	99.10	99.11	+0.01	-

Paper values are from Table 6 / Table 7 of the published paper (arXiv 2207.13159). 2,048 test pairs, LEVIR-CD-256 split of 7,120 / 1,024 / 2,048 (train / validation / test).

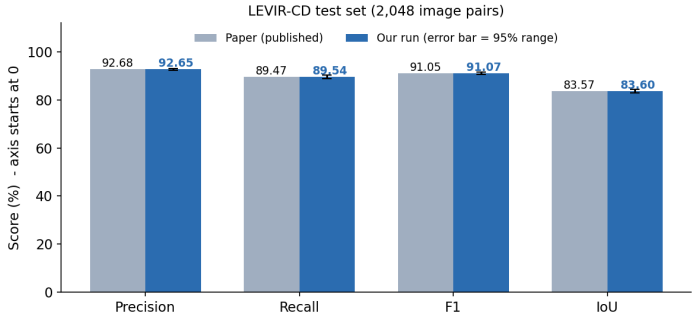


Figure 3 - Paper (grey) against our run (blue). The bars are the same height to within a hair. The black whiskers show the 95% give-or-take: the paper's value sits inside our whiskers for every score. The vertical axis starts at zero so the picture is not exaggerated.

How do we know it is a genuine reproduction?

- **Official code, official file.** The model weights came from the authors' repository (checked with a fingerprint, SHA-256 743645b8…4a3e). The test program is the authors' own; the only change was a one-line compatibility fix for a newer version of Python, with no change to the maths.
- **Official data format.** 256 × 256 tiles, 2,048 test pairs - the numbers the paper states. Our image loading matches the official loader exactly (difference = 0.0).
- **Two independent runs agree.** The authors' script and our own evaluation script (which adds the error bars) produced the same F1 down to 16 decimal places: 0.9106975154790973.
- **No peeking.** We did not tune any setting on the test pictures. The 0.5 cut-off is the authors' value, and the sweep we ran on separate *validation* pictures found that 0.5 was already almost optimal (best 0.44, F1 90.85 vs 90.80 at 0.5).

So is 91.07 'better' than the paper's 91.05?

No - they are the same result. A difference of 0.02 points is about **twenty times smaller** than the natural wobble (±0.4) shown by the error bars. Differences this small come from ordinary things such as a different graphics card or library version. We would only claim an *improvement* if our whole range sat clearly above the paper's number; it does not. The right conclusion is that **the paper's claim holds up when somebody else re-runs it**.

4. What the results look like

Beyond the headline score, we ran the pretrained model on a small, randomly chosen sample (seed 0) of **20 test pairs** (14 with changes and 6 labelled as 'nothing changed') and compared it with two simple methods that just subtract the colours of the two photos. These are the pictures from our notebook.

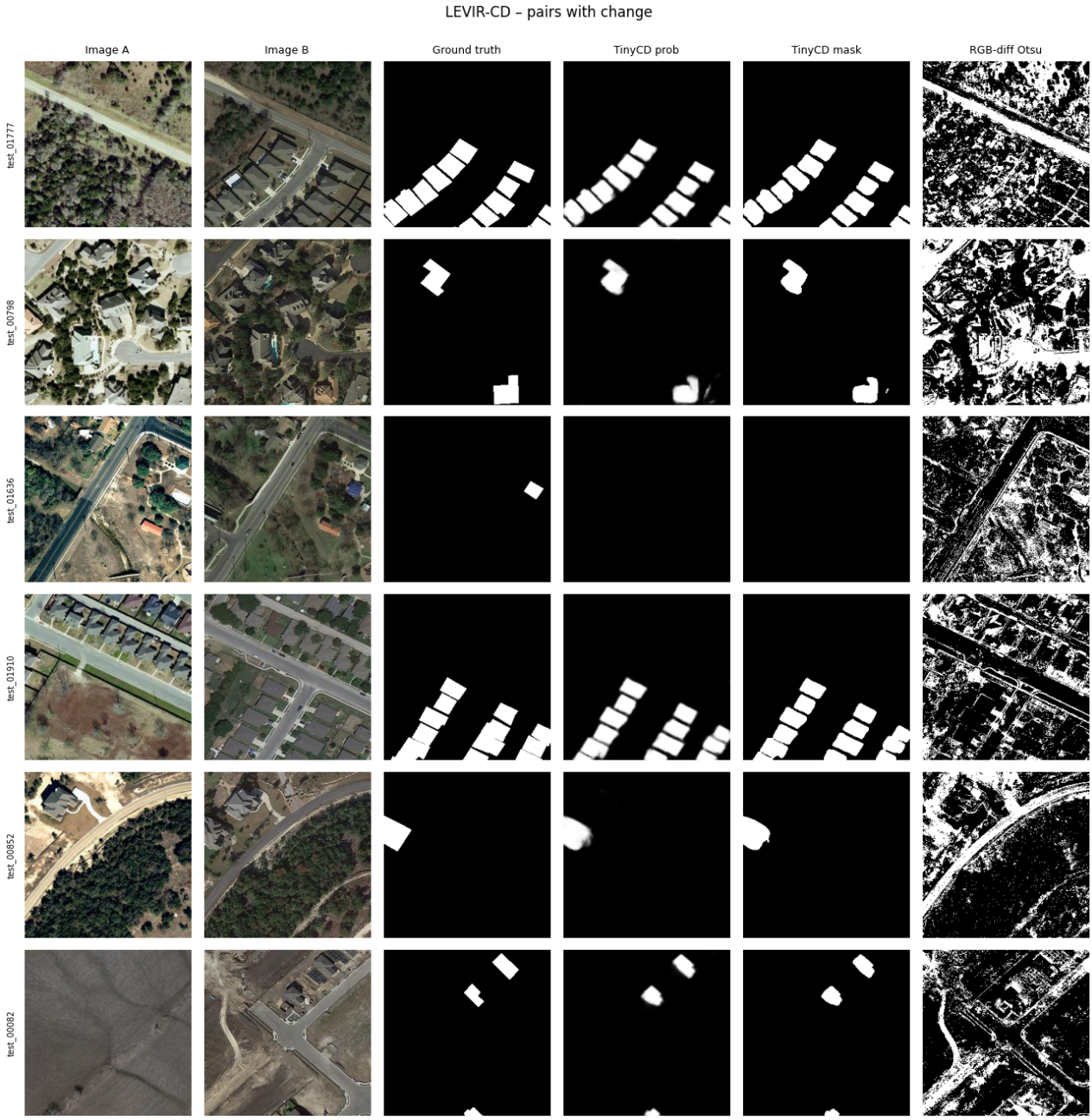


Figure 4 - Six pairs with changes. Columns: before, after, human answer, model probability, model mask, and the simple colour-difference method. TinyCD (columns 4-5) tracks the human answer; the simple method (last column) lights up roads, trees and shadows. Row 3 shows a real miss: one small building was not found at all.

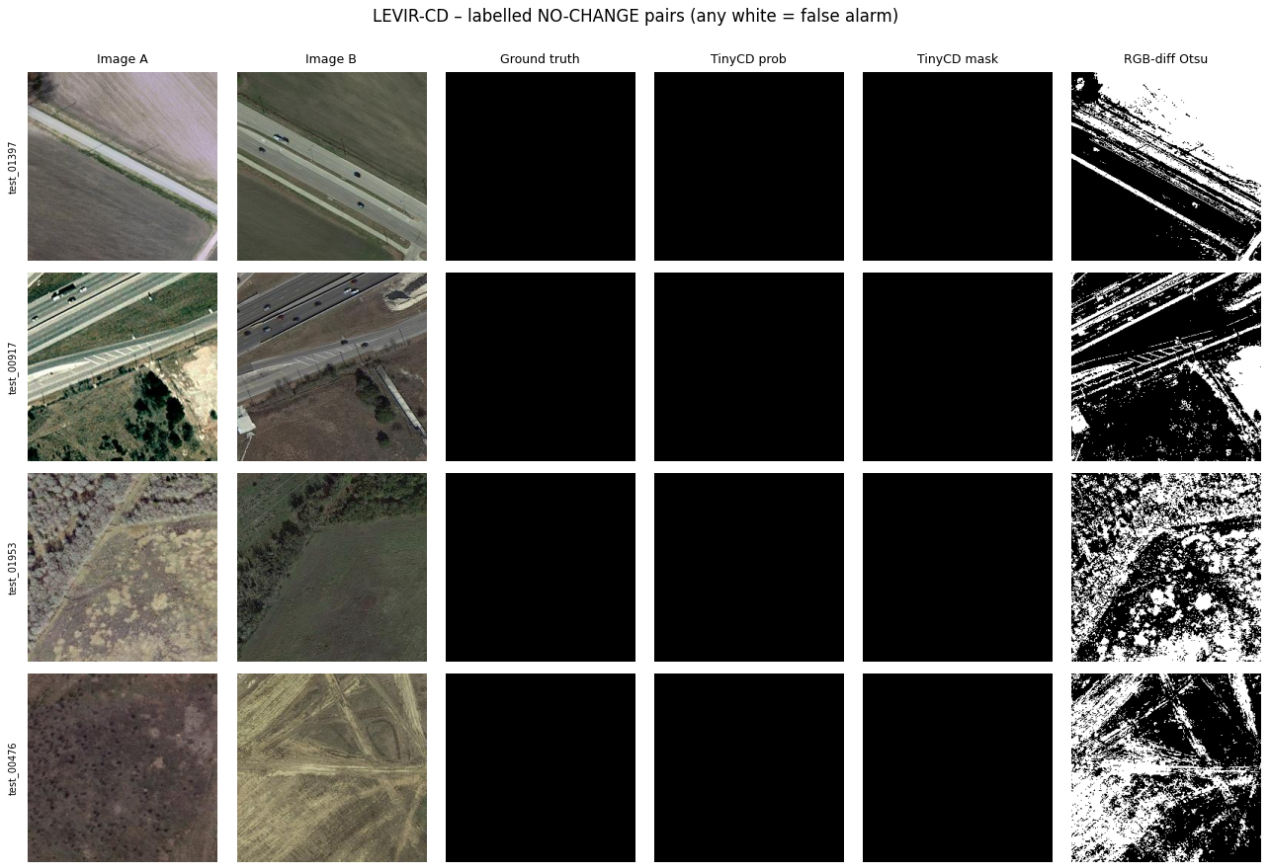


Figure 5 - Four pairs where, according to the human labels, nothing changed. A good model leaves the mask black. TinyCD stays black on these; the simple method paints a large part of the picture white (false alarms).

Comparison with the simple colour-difference methods

Method (same 20 pairs)	Precision	Recall	F1	IoU	'No change' pairs with false alarm
TinyCD (pretrained)	0.926	0.864	**0.894**	0.808	**0 of 6**
Colour difference, fixed cut-off (103)*	0.081	0.125	0.099	0.052	6 of 6
Colour difference, Otsu automatic cut-off	0.064	0.327	0.107	0.056	6 of 6

**The fixed cut-off was chosen on the separate validation set (1,024 pairs), not on these pairs. With only 20 pairs these numbers are a sanity check, not a benchmark; the full 2,048-pair result is on page 4.*

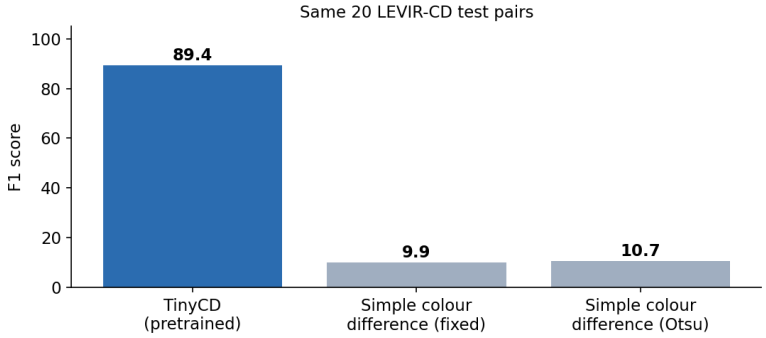


Figure 6 - On the same 20 pairs, TinyCD scores about nine times higher than simply subtracting the two photos.

5. What it does well and where it struggles

Using all 2,048 test pairs, we also broke the score down by the size of the change. This answers the question "does it catch small things?"

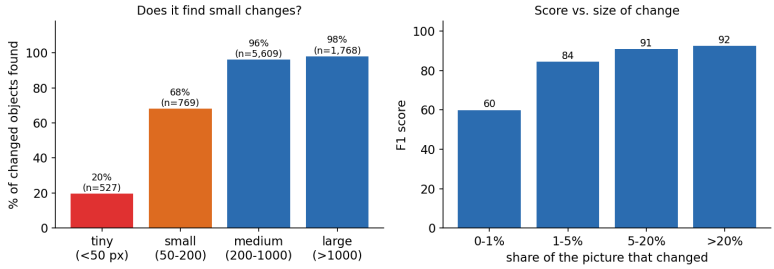


Figure 7 - Left: share of individual changed objects the model found, by object size (a changed object counts as found if at least half of it is covered). Right: F1 score depending on how much of the picture changed. 1 pixel = 0.5 metres on the ground, so 50 pixels is roughly a garden shed.

Reliable

- **Normal-sized buildings that appear or disappear.** 96-98% of objects larger than 200 pixels are found, with F1 above 90 when the change covers more than 5% of the picture.
- **Not crying wolf.** On the 1,113 test pairs that contain no change at all, only 33 (3.0%) had even a single false-alarm pixel, and only 0.011% of those pictures' pixels were wrongly painted white.
- **Ignoring colour and shadow changes** that fool the simple method (Figures 4 and 5).

Struggles

- **Very small changes.** Objects under 50 pixels are found only 20% of the time, and 50-200 pixels about 68%. When only 0-1% of the picture changes, F1 drops to about 60. *Trade-off:* the model prefers to miss a tiny change rather than raise a false alarm.
- **A different dataset (SYSU-CD).** Using the same model on 4,000 SYSU-CD pairs it had never seen, F1 collapsed to 0.003 and it found only 0.16% of the changes. Two reasons: the model was unsure on new imagery (its most confident pixel in a picture averaged only 0.14 out of 1), and SYSU-CD's human answers are rough shaded regions, not building outlines. This is a warning about moving to new data, not a fair score for the model (Figure 8).

The cross-dataset test (SYSU-CD) in pictures

SYSU-CD is a different collection of aerial photos (Hong Kong, 2007-2014). The model was never trained on it. We ran the LEVIR-trained model on all 4,000 SYSU-CD test pairs unchanged (F1 0.003; precision 82%, recall 0.16%): when it does paint something it is usually right, but it stays silent almost everywhere. A threshold sweep on SYSU validation pictures was **not** run, so we cannot say whether a lower cut-off would have helped.

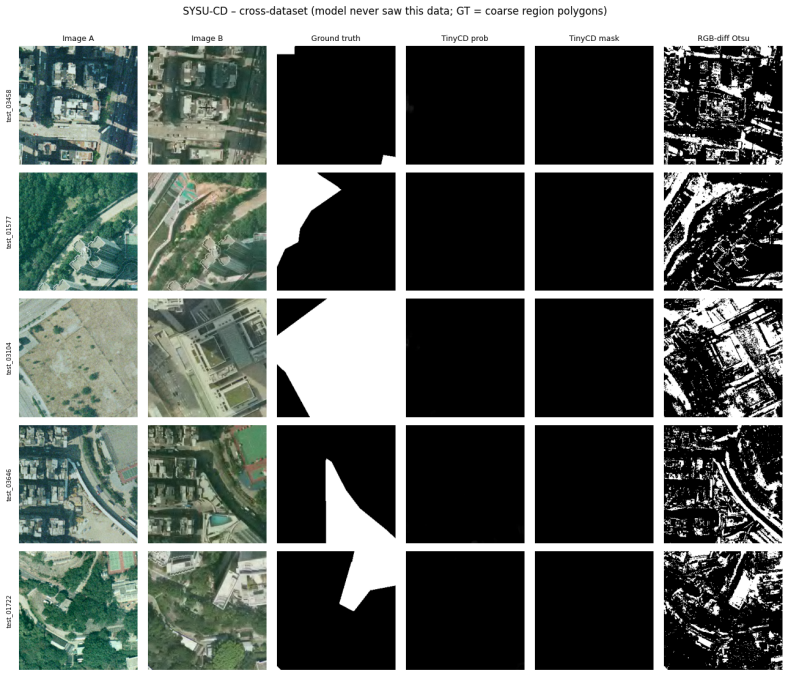


Figure 8 - Cross-dataset test on SYSU-CD. The model does react to some new buildings (bright patches in column 4) but weakly, and the human-drawn answers (column 3) are coarse blobs. On this data even the simple colour method scored higher (F1 0.45 vs 0.00 on a 10-pair sample).

6. Limits, licence and next steps

What we did not do, and why

- **No retraining.** We reproduced the *test* result of the released model. Whether the paper's *training* recipe would give the same file again was not checked.
- **The second paper dataset (WHU-CD, paper F1 91.74) was not reproduced.** The official download page blocks automated downloads, and the authors' pre-processed copy on Google Drive returned an access-denied error (HTTP 401). The authors split WHU-CD *randomly*, so a split we made ourselves could contain tiles the model trained on, which would make the score meaningless. The WHU file (**whu_best.pth**) is in the repository but was not tested.
- **No test on ordinary photos, products, text or screenshots.** The model was only trained and tested on aerial building changes. Nothing here shows it works elsewhere.
- **Not tested on misaligned or badly lit pairs.** The authors say it assumes the two photos line up. Also unknown: how the authors chose the 'best' saved model (validation or test pictures) - a small caveat on the published number.

Licence - important for a client project

The TinyCD code and weights are released **"for non-commercial and research purposes only"**; there is no separate licence file and the authors ask commercial users to contact them. The LEVIR-CD pictures may be used for **academic purposes only**. SYSU-CD states no licence. **This work is therefore not cleared for a commercial client prototype** until the authors give written permission. Using it for an internal research demo is fine.

Recommended next steps

- **Decide the client's real pictures.** If they are aerial or satellite photos of buildings, TinyCD is a sensible starting point. If they are product photos, documents or app screenshots, this model was not built for that and would need new labelled examples.
- **If performance is not enough:** collect a small set of the client's own labelled pairs (even 100-200), test the unchanged model on them, and only then consider fine-tuning.
- **If commercial use matters:** email the authors for permission, or pick a model with a permissive licence and repeat this same test.
- **Finish WHU-CD** once the authors' pre-processed copy (or a verified mirror) is available.

Reproduce it yourself

Project folder **image difference/**: notebook **notebooks/tinycd_demo.ipynb**, **scripts/evaluate.py** (full test with error bars), **results/** (logs and numbers quoted here; the official-script run is **results/logs/official_test_levir_on_levir.log**) and the README.

Setup used	
Computer	NVIDIA GeForce GTX 1080 Ti (11 GB), Linux, Python 3.10.12
Software	PyTorch 2.7.1 (CUDA 11.8); official TinyCD code commit 1696827 (12 Jan 2024)
Model file	levir_best.pth, 1,276,499 bytes, SHA-256 743645b87199c96b...4a3e
Data / timing	LEVIR-CD 256 × 256 tiles from the Hugging Face mirror ericyu/LEVIRCD_Cropped256 (2,048 test pairs, matches the paper's split sizes); SYSU-CD from ericyu/SYSU_CD. 18.4 ms per pair (model only); whole 2,048-pair test in 32 s.